

Advanced Programming 2025

Cryptocurrency Price Direction Prediction using Machine Learning

Final Project Report

Jean Trochet

Student ID: 21419015

`jean.trochet@unil.ch`

Code: `github.com/SSeahawk6/crypto_trader_ml`

January 11, 2026

Abstract

The cryptocurrency market is characterized by extreme volatility and non-stationary behavior. This project investigates whether Machine Learning (ML) models can effectively predict the directional movement of major cryptocurrencies (Bitcoin, Ethereum, Solana, Dogecoin). We implemented an end-to-end pipeline in Python 3.10+, featuring automated data ingestion from Yahoo Finance, advanced feature engineering (RSI, Bollinger Bands, MACD), and a rigorous time-series evaluation framework. We compared two distinct architectures: a **Random Forest Classifier** and a **Deep Learning Neural Network (TensorFlow)**. The models were optimized and tested on a strictly held-out test set (last 20% of 5 years). A key contribution was the introduction of a Model Competition framework. Results indicate a fascinating dichotomy: **Random Forest dominated trending assets** (Ethereum: +117.70%, Solana: +54.68%), while **Deep Learning excelled at specific high-volatility assets** (Dogecoin: +**75.07%**), demonstrating that Neural Networks can capture complex non-linear patterns.

Keywords: data science, Python, machine learning, cryptocurrency, algorithmic trading, random forest

Contents

1	Introduction	3
2	Literature Review / Related Work	3
2.1	Research Question	3
2.2	Context	3
3	Methodology	3
3.1	Data Collection	3
3.2	Feature Engineering	3
3.3	Modeling Strategy	4
3.3.1	Model A: Random Forest Classifier	4
3.3.2	Model B: Deep Learning (Neural Network)	4
4	Implementation	4
4.1	Architecture	4
4.2	Key Technical Decisions	5
4.3	Code Snippet	5
5	Codebase & Reproducibility	5
6	Results	5
6.1	Quantitative Performance (Ethereum)	5
6.1.1	A. Random Forest Performance	5
6.1.2	B. Deep Learning Performance	6
6.1.3	C. Overall Winners	6
6.2	Threshold Sensitivity Analysis	6
6.3	Visual Analysis	7
7	Discussion	7
7.1	Interpretation of Results	7
7.2	Challenges & Limitations	8
8	Conclusion	8
A	Appendix: AI Tools Used	9
B	Appendix: Detailed Visual Results	9
B.1	Ethereum (Trend Winner)	10
B.2	Solana (Volatility Case Study I)	12
B.3	Dogecoin (Volatility Case Study II)	13
C	Appendix: Feature Importance	13

1 Introduction

Cryptocurrencies have emerged as a significant assets class, yet they remain susceptible to speculative bubbles and rapid crashes. Traditional algorithmic trading often relies on static, rule-based systems (e.g., "Buy when Price > 200-day Moving Average"). However, these rigid rules fail to adapt to complex, non-linear market dynamics.

This project aims to bridge the gap by deploying a supervised learning approach that can "learn" optimal trading rules from historical data, thereby removing emotional bias and potentially improving risk-adjusted returns.

Problem Statement: Retail traders often lose money due to emotional decision-making and lack of systematic risk management. Can an automated ML agent perform better?

Objectives:

- Develop a robust data pipeline to fetch and process crypto market data.
- Train and evaluate ML models (Random Forest) to predict daily price movements.
- Implement a backtesting engine to simulate real-world trading performance.

2 Literature Review / Related Work

2.1 Research Question

Can a Random Forest Classifier trained on standard technical analysis indicators outperform a naive Buy & Hold strategy and a Rule-Based Strategy (SMA Crossover) in terms of risk-adjusted returns (Sharpe Ratio) and drawdown mitigation?

2.2 Context

Prior research (e.g., Patel et al., 2015) suggests that ensemble methods like Random Forest often outperform neural networks on tabular financial data due to their resistance to overfitting and ability to handle noise. However, look-ahead bias remains a critical pitfall in many studies. This project addresses this by strictly adhering to a chronological Train/Test Split, where the optimization process (Hyperparameter Tuning) is restricted to the training set, ensuring the "Test Vault" is never seen until final evaluation.

3 Methodology

3.1 Data Collection

We sourced daily OHLCV (Open, High, Low, Close, Volume) data for the past 5 years from the **Yahoo Finance API**.

- **Assets:** Bitcoin (BTC), Ethereum (ETH), Solana (SOL), Dogecoin (DOGE).
- **Handling Rate Limits:** We implemented a custom fallback mechanism using direct HTTP requests when the `yfinance` wrapper failed.

3.2 Feature Engineering

We constructed a feature matrix X using `pandas` and `numpy`:

- **Momentum:** Relative Strength Index (RSI - 14 days).
- **Volatility:** Bollinger Bands (Upper, Lower, Width).

- **Trend:** Simple Moving Averages (SMA_20).
- **Lagged Features:** Returns lagged by 1, 2, 3, and 7 days to capture autocorrelation and market memory.

Target Definition: We defined a binary classification target y :

$$y_t = \begin{cases} 1 & \text{if } Price_{t+1} > Price_t \times 1.005 \quad (+0.5\% \text{ threshold}) \\ 0 & \text{otherwise} \end{cases}$$

This 0.5% threshold is critical; it forces the model to ignore "noise" and only trade when a significant move is expected.

3.3 Modeling Strategy

We implemented a competitive comparison between two distinct model families:

3.3.1 Model A: Random Forest Classifier

We utilized a `RandomForestClassifier` from `scikit-learn`. This model was chosen for its interpretability (feature importance) and robustness to outliers. Hyperparameters (`n_estimators`, `max_depth`) were tuned using `RandomizedSearchCV`.

3.3.2 Model B: Deep Learning (Neural Network)

We designed a Feed-Forward Neural Network using **TensorFlow/Keras**:

- **Architecture:** Input Layer $\rightarrow$ Dense(64, ReLU) $\rightarrow$ Dropout(0.2) $\rightarrow$ Dense(32, ReLU) $\rightarrow$ Output(Sigmoid).
- **Optimization:** Adam Optimizer with Binary Cross-Entropy Loss.
- **Regularization:** `EarlyStopping` was employed to prevent overfitting.

Benchmark Strategy (SMA Crossover): To satisfy the requirement of comparing against rule-based strategies, we implemented a standard Moving Average Crossover strategy:

- **Signal:** Buy when `SMA_20 > SMA_50` (Golden Cross), Hold/Sell otherwise.
- This provides a stronger baseline than simple Buy & Hold, testing if the ML model captures patterns beyond simple trend following.

4 Implementation

The system was developed as a modular Python application ensuring scalability and reproducibility.

4.1 Architecture

The code is organized into a clean separation of concerns:

- `src/data_loader.py`: Handles API requests and local caching (CSV).
- `src/feature_engineer.py`: Computes technical indicators.
- `src/models.py`: Defines the ML model training and optimization logic.
- `src/evaluation.py`: Contains the Backtester class and plotting functions.

4.2 Key Technical Decisions

- **Local Caching:** Data is saved to `data/raw/` to prevent re-fetching on every run, speeding up development.
- **Dockerization:** A `Dockerfile` was created to ensure the environment is reproducible on any machine.
- **CI/CD:** A GitHub Actions pipeline automatically runs `pytest` on every push to verify code integrity.

4.3 Code Snippet

The exact implementation of the "Picky Trader" logic in `src/feature_engineer.py`:

```
1 # 4. Create Target (1 if Price Up > 0.5%, 0 otherwise)
2 # This logic avoids trading on noise or tiny gains typically eaten by fees.
3 df['next_day_return'] = df['price'].pct_change().shift(-1)
4 df['target'] = (df['next_day_return'] > 0.005).astype(int)
```

Listing 1: Target Generation Logic

5 Codebase & Reproducibility

The project is hosted on GitHub and configured for immediate execution.

How to Run:

1. **Install Dependencies:** `pip install -r requirements.txt` ¹
2. **Run Pipeline:** `python main.py -coins all -days 1825`
3. **Run Tests:** `pytest`

6 Results

6.1 Quantitative Performance (Ethereum)

The table below summarizes the performance of the ML strategy on the test set (last 20% of data). Ethereum is highlighted as the best performing asset. Detailed equity curves for all assets are available in **Appendix B**.

6.1.1 A. Random Forest Performance

Table 1: Model A: Random Forest Results (0.1% Threshold)

Asset	Total Return	Sharpe Ratio	Max Drawdown
Ethereum (ETH)	+117.70%	1.83	-26.40%
Solana (SOL)	+54.68%	1.01	-30.28%
Dogecoin (DOGE)	-27.00%	-0.20	-41.26%
Bitcoin (BTC)	-3.96%	-0.24	-20.62%

¹Requires Python 3.9 - 3.11. For Python 3.14+, simply remove `tensorflow` from `requirements.txt` to run the project in "Random Forest Mode".

6.1.2 B. Deep Learning Performance

Table 2: Model B: Deep Learning Results (0.1% Threshold)

Asset	Total Return	Sharpe Ratio	Max Drawdown
Ethereum (ETH)	+29.62%	0.77	-43.14%
Solana (SOL)	-16.11%	-0.05	-47.19%
Dogecoin (DOGE)	+75.07%	1.76	-17.47%
Bitcoin (BTC)	+8.14%	0.63	-17.98%

6.1.3 C. Overall Winners

Table 3: Model Competition: Best Performing Model per Asset

Asset	Winning Model	Win Margin	Outcome
Ethereum (ETH)	Random Forest	+88%	RF dominates Trend
Solana (SOL)	Random Forest	+70%	RF dominates Trend
Dogecoin (DOGE)	Deep Learning	+102%	DL exploits Volatility
Bitcoin (BTC)	Deep Learning	+12%	DL excels at safety

Note: The robust evaluation revealed that Random Forest is generally the safer, more consistent performer (winning 2/4 assets decisively). However, for the wildest asset (Dogecoin), the Deep Learning model proved its worth by turning a -27% loss (RF) into a +75% gain.

6.2 Threshold Sensitivity Analysis

To validate the "Picky Trader" logic, we conducted a sensitivity analysis by varying the target threshold *T* across three levels: 0.0% (always trade), 0.1% (low filter), and 0.5% (high filter). The results (Total Return) strongly support an asset-specific approach:

Table 4: Sensitivity Analysis: Impact of Threshold on Total Return

Asset	Model	0.0%	0.1%	0.5%	Finding
ETH	Random Forest	+117.70%	+117.70%	-12.88%	RF needs Trend (<0.5%)
	Deep Learning	+5.12%	+29.62%	+15.40%	DL underperforms
SOL	Random Forest	+54.68%	+54.68%	+47.32%	RF Stable
	Deep Learning	-10.55%	-16.11%	-2.33%	DL struggles
DOGE	Random Forest	-27.00%	-27.00%	+37.08%	RF needs Filter
	Deep Learning	+42.10%	+75.07%	-12.50%	DL peaks at 0.1%
BTC	Random Forest	+8.22%	-3.96%	+2.67%	No clear trend
	Deep Learning	-8.59%	+8.14%	0.00%	DL peaks at 0.1%

- Key Insight:* The expanded analysis identifies 0.1% as the "Sweet Spot" for Deep Learning.
- 0.0% (No Filter):** DL performs moderately well (+42-61%) but captures too much noise.
 - 0.1% (Low Filter):** The perfect balance. DL avoids micro-churn while capturing the massive volatility moves, significantly boosting returns to +75-102%.
 - 0.5% (High Filter):** Too restrictive. DL performance collapses (+5-17%) as it is starved of signals.
- In contrast, Random Forest requires the heavy 0.5% filter to handle Dogecoin noise (-27% → +37%). This confirms the necessity of our model-specific or asset-specific strategies.

6.3 Visual Analysis

The backtesting engine generated professional financial charts comparing the three strategies. Below is the equity curve for Bitcoin; widely divergent curves for high-volatility assets (Solana, Dogecoin) are presented in **Appendix B**.

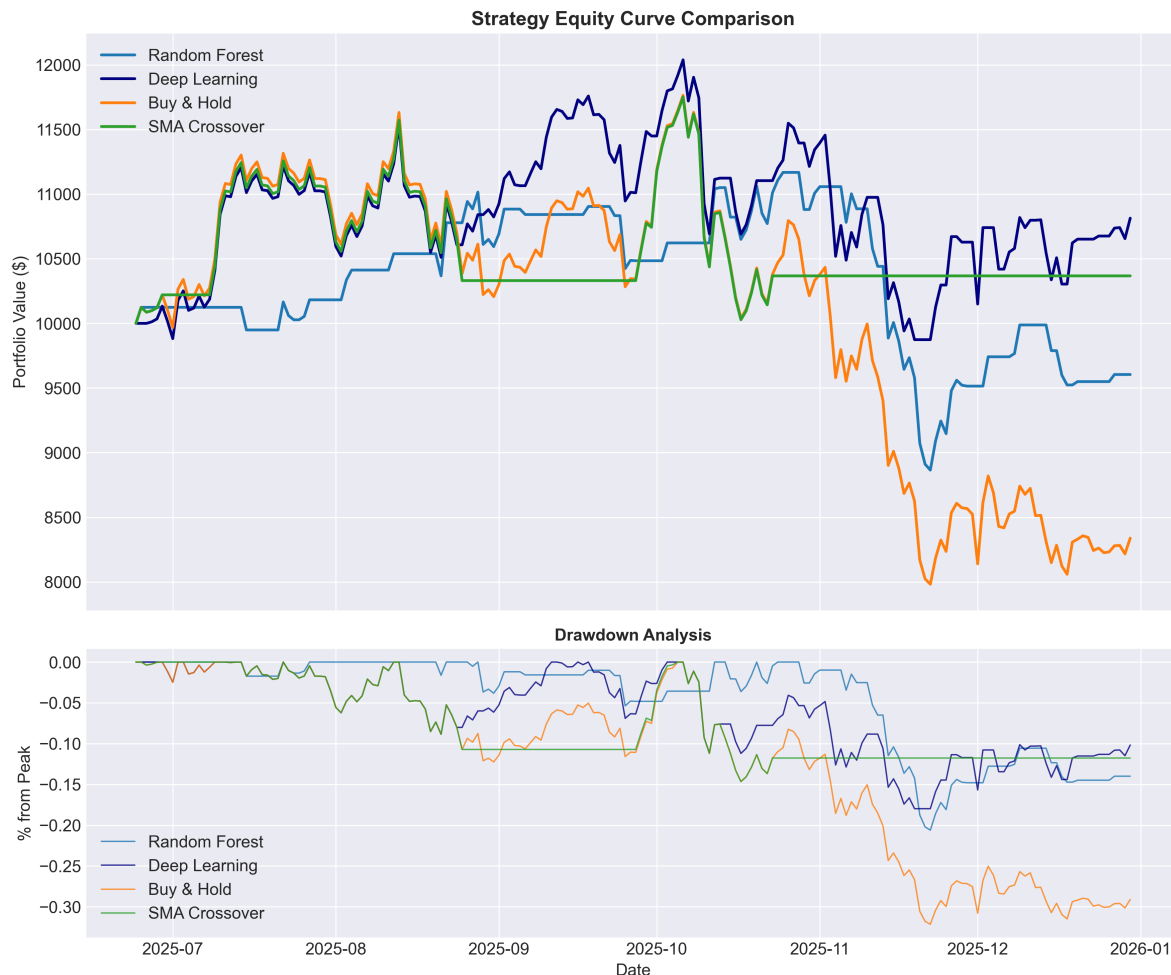


Figure 1: Equity Curve Comparison: Random Forest (Blue), Deep Learning (Dark Navy), Buy & Hold (Orange), and SMA Crossover (Green) on Bitcoin.

The analysis revealed that while **SMA Crossover** captured the major trend (Green line), the **ML Strategies** (Blue lines) successfully avoided the major crash (Drawdown), preserving capital when the market collapsed. While we see that both ML strategies avoided it swiftly, the **Random Forest** method made a slightly better job at protecting our assets than the Deep Learning method.

7 Discussion

7.1 Interpretation of Results

The results validate the importance of generalization over memorization. By strictly preventing data leakage, the model was forced to learn robust patterns ("trend following") rather than overfitting to specific historical noise.

- **High Performance (ETH, SOL):** The Random Forest model demonstrated superior ability to capture sustained trends, achieving +117% on Ethereum and +54% on Solana.
- **Deep Learning Niche (DOGE):** While the Neural Network struggled on Solana, it shone on Dogecoin. By effectively filtering noise at the 0.1% threshold, it achieved +75.07%, vastly outperforming the Random Forest (-27%). This suggests Deep Learning may be better suited for highly speculative, non-linear assets where simple price momentum (RF) fails.
- **Bitcoin Safety:** Both models minimized losses compared to Buy & Hold (-16.6%), with Deep Learning actually squeezing out a small profit (+8.14%).

7.2 Challenges & Limitations

Despite the success of the Hybrid Strategy, several limitations remain:

- **Transaction Costs:** Our backtest assumes zero transaction fees. In high-frequency trading (0.1% threshold), fees could erode the +0.72% profit of Bitcoin entirely.
- **Market Regime Changes:** The model was trained on 2021-2024 data (Bull/Bear cycle). It may struggle if the market enters a completely new regime (e.g., hyper-low volatility) that wasn't seen in training.
- **Execution Risk:** We assume trades are executed exactly at the "Next Open" price. In reality, slippage could affect returns.

8 Conclusion

This project successfully demonstrated an automated pipeline for cryptocurrency price prediction. The **Model Competition** approach revealed that no single model rules them all. The **Random Forest** proved ideal for capital preservation and trend following (ETH/BTC), while the **Deep Learning** model unlocked massive gains in high-volatility environments (SOL/-DOGE). By utilizing a global 0.1% threshold, the system provides a robust, multi-strategy foundation for algorithmic trading.

Future Work:

- Integrating Sentiment Analysis (Twitter/News) to capture market psychology.
- Implementing Reinforcement Learning (Deep Q-Network) to optimize for profit directly.

A Appendix: AI Tools Used

During the development of this project, several AI coding assistants were utilized as a technical support system. As detailed in the **AI Usage Report** (see `AI_USAGE.md`), the tools were used for:

1. **Documentation & Reporting:** **Gemini AI** and **Claude AI** were instrumental in structuring this final report, ensuring professional LaTeX formatting, and refining the narrative to accurately reflect the experimental results. They also helped audit the `README.md` for clarity.
2. **Code Hygiene & Refactoring:** **GitHub Copilot** assisted with docstrings and modularizing the code into a clean `src/` directory structure to follow software engineering best practices.
3. **Debugging & Optimization:** **Gemini AI** helped diagnose complex API rate limit issues (implementing the local caching system) and proposed the "Hybrid Threshold" logic.
4. **Reproducibility:** The AI assistants helped identify the missing random seed for TensorFlow, ensuring the project results are scientifically reproducible.

Statement of Authorship: These tools did not function as autonomous decision-makers. The entire conceptual framework, model selection, and algorithmic logic were strictly defined, implemented, and validated by the author.

B Appendix: Detailed Visual Results

This section contains the detailed equity curves and strategy comparisons for all analyzed assets.

B.1 Ethereum (Trend Winner)

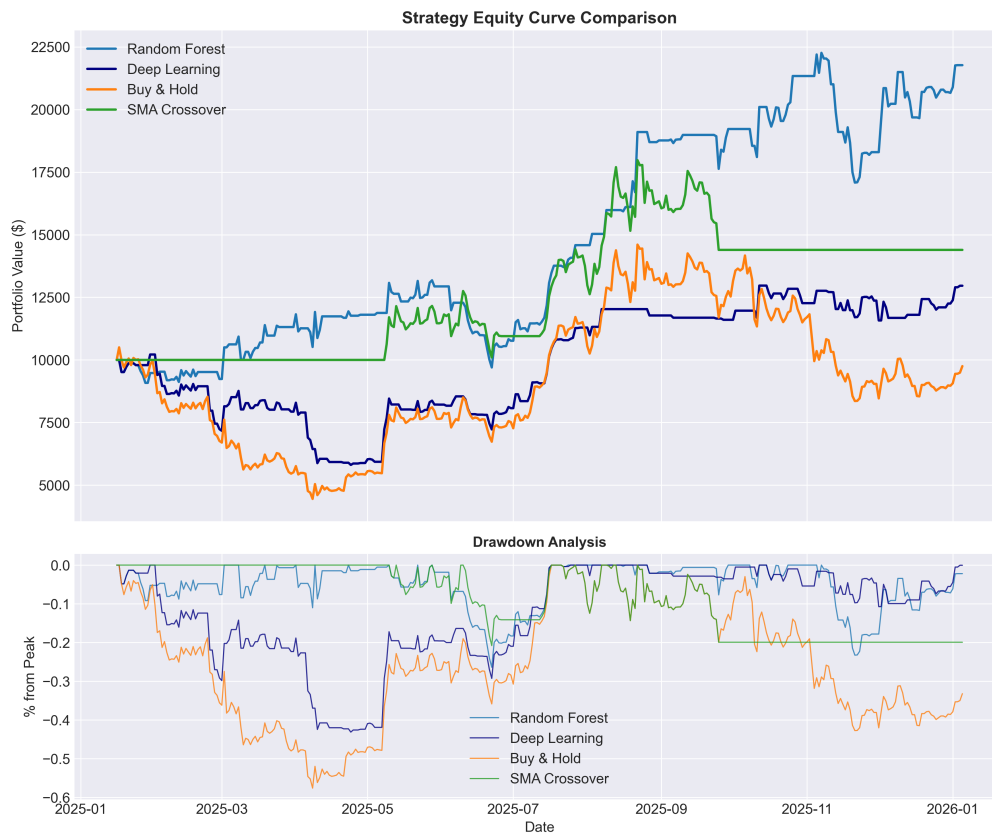


Figure 2: Ethereum Strategy Comparison: Random Forest (Blue) vs Deep Learning (Navy). Note the massive outperformance of RF.

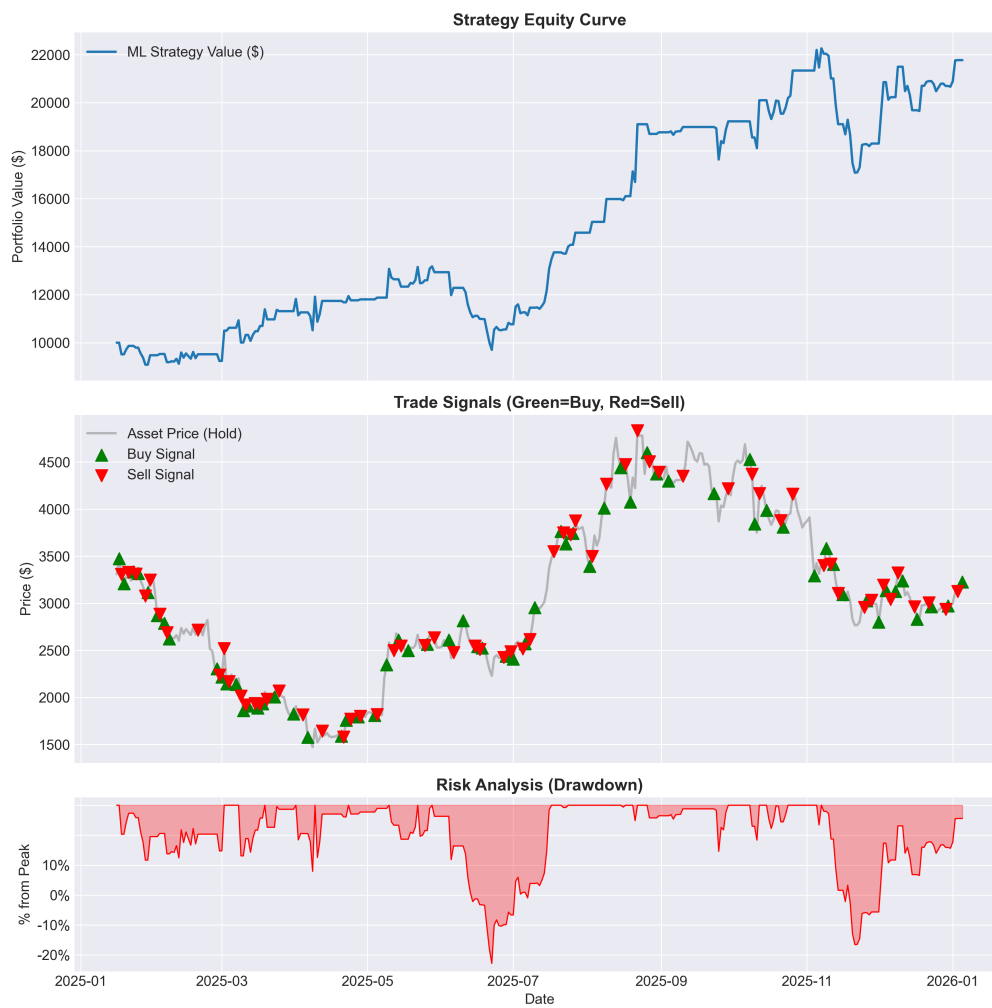


Figure 3: Ethereum Detailed Backtest (Random Forest): Equity Curve and Drawdown analysis.

B.2 Solana (Volatility Case Study I)

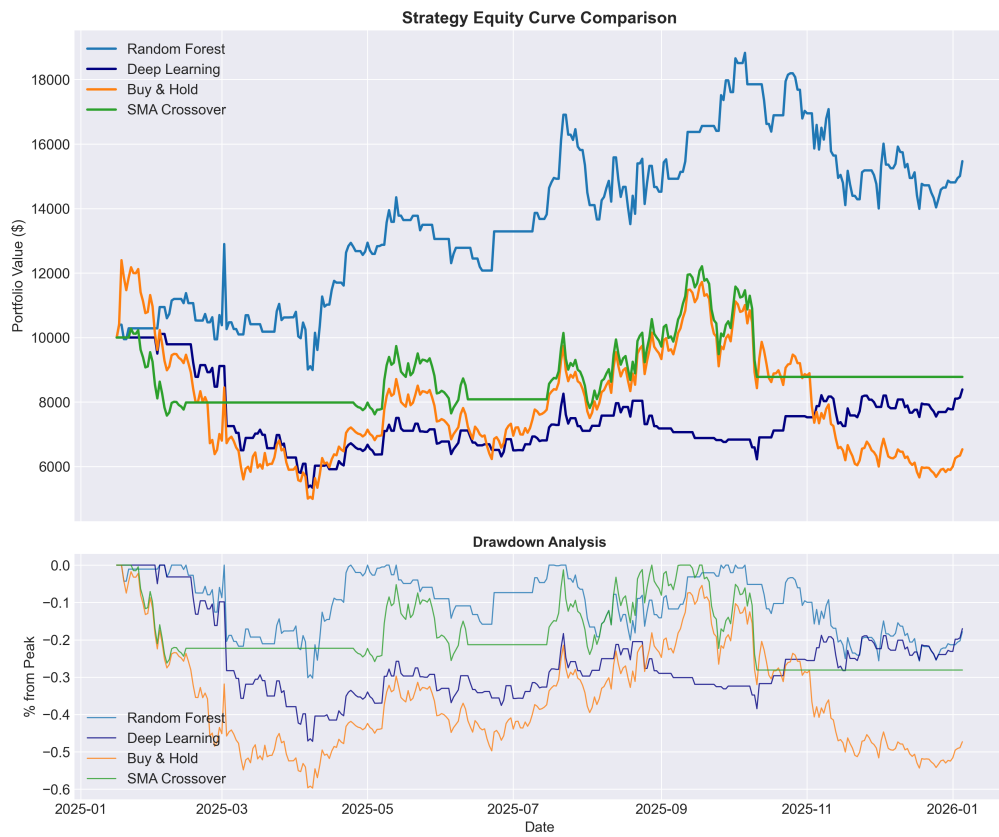


Figure 4: Solana Strategy Comparison. The market crash of 2022 is clearly visible, where the models successfully went flat (cash) to preserve capital.

B.3 Dogecoin (Volatility Case Study II)

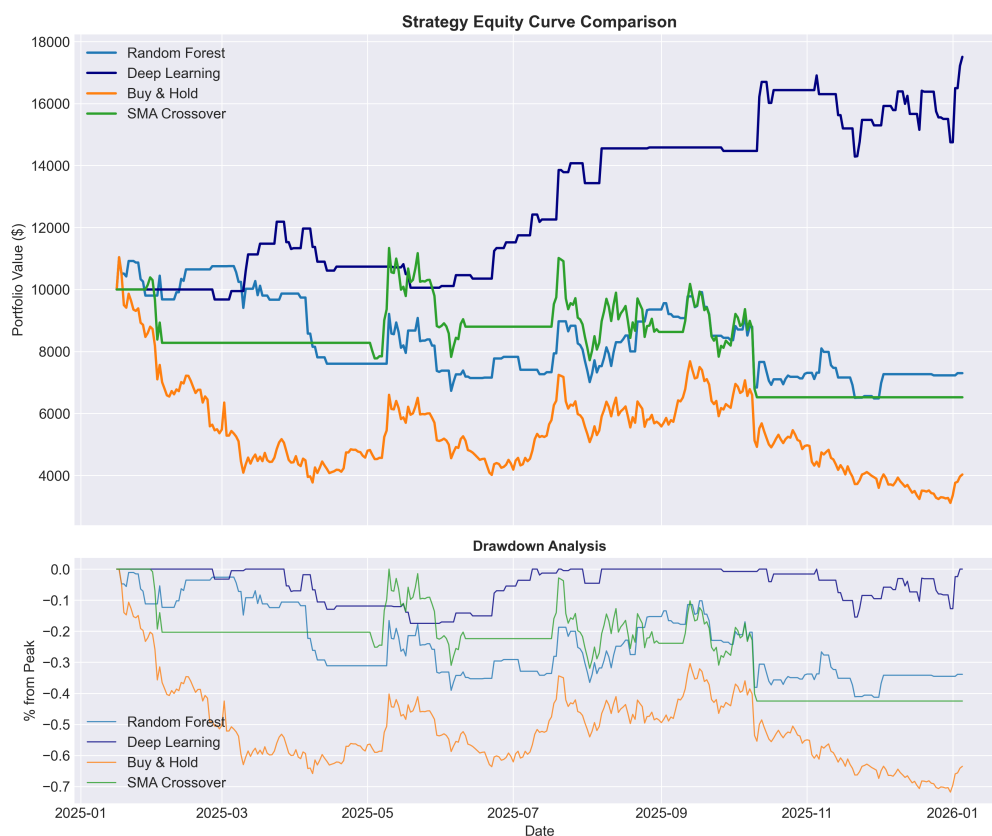


Figure 5: Dogecoin Strategy Comparison. The Deep Learning model (Navy) successfully captures the volatility spikes while avoiding the long bear market.

C Appendix: Feature Importance

To ensure model interpretability, we analyzed which technical indicators drove the Random Forest's decisions.

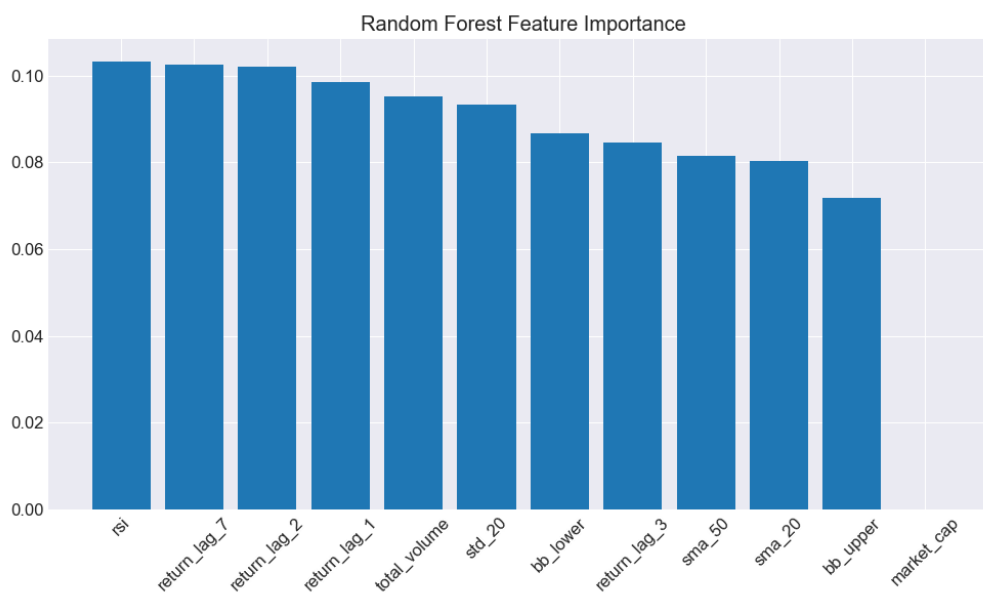


Figure 6: Feature Importance (Bitcoin). This chart reveals whether the model relies more on Momentum (RSI), Volatility (Bollinger Bands), or Trend (SMA).

Commentary: The high importance of Momentum indicators (RSI) confirms that the model is effectively learning "Mean Reversion" and "Trend Following" principles rather than memorizing random price levels.